

Proyecto: Diseño de un sistema mediante Programación Orientada a Objetos

1. Objetivo general

Diseñar e implementar en Java un pequeño modelo orientado a objetos que represente las principales entidades y relaciones de un videojuego de tipo **RPG (Role-Playing Game)**.

El proyecto deberá demostrar el uso correcto de los principales conceptos de Programación Orientada a Objetos:

- Clases y objetos.
- Encapsulamiento.
- Herencia.
- Clases abstractas.
- Interfaces.
- Polimorfismo.
- Composición.
- Agregación.

2. Contexto del proyecto

El juego se desarrolla en un mundo de fantasía donde diferentes personajes pueden combatir, utilizar objetos, desarrollar habilidades, participar en misiones y colaborar entre sí.

El sistema deberá permitir representar diferentes tipos de personajes, cada uno con características y comportamientos propios, pero compartiendo una estructura común.

Por ejemplo:

- Un **Guerrero** puede atacar utilizando una espada.
- Un **Mago** puede lanzar hechizos utilizando maná.
- Un **Arquero** puede realizar ataques a distancia utilizando un arco.
- Un **Sacerdote** puede curar a otros personajes.
- Todos los personajes poseen un inventario.

- Los inventarios contienen diferentes tipos de objetos.
- Una misión puede involucrar a varios personajes.
- Una misión continúa existiendo independientemente de que alguno de los personajes deje de participar en ella.

3. Clase abstracta Character

Se deberá crear una clase abstracta denominada:

Character

Esta clase representa la estructura común de todos los personajes del juego.

Atributos mínimos

- name — nombre del personaje.
- level — nivel.
- health — puntos de salud.
- maxHealth — salud máxima.
- inventory — inventario del personaje.

Se podrán agregar otros atributos, por ejemplo:

- experience
- strength
- defense
- status

Métodos principales

La clase deberá definir comportamientos comunes y métodos abstractos.

Ejemplo:

- attack(Character target)
- defend(int damage)
- levelUp()

- `isAlive()`
- `showStats()`

Los métodos `attack()` y `defend()` deberán ser abstractos, ya que cada tipo de personaje puede implementar estas acciones de manera diferente.

4. Herencia: tipos de personajes

De la clase `Character` deberán heredar al menos las siguientes clases:

Warrior

Representa a un guerrero.

Características:

- Utiliza armas de combate cuerpo a cuerpo.
- Tiene una mayor capacidad de defensa.
- Puede realizar ataques físicos.

Ejemplo de comportamiento:

`attack(Character target)`

realiza un ataque utilizando la fuerza del personaje y el arma equipada.

Mage

Representa a un mago.

Características:

- Posee puntos de maná.
- Puede utilizar habilidades mágicas.
- Sus ataques pueden consumir maná.

Atributo adicional:

- `mana`
- `maxMana`

Métodos posibles:

- `castSpell(Character target)`
- `restoreMana()`

Archer

Representa a un arquero.

Características:

- Utiliza arco y flechas.
- Puede realizar ataques a distancia.
- Puede disponer de una cantidad limitada de flechas.

Atributos adicionales:

- `arrows`
- `range`

Métodos posibles:

- `shoot(Character target)`
- `reload()`

Priest

Representa a un sacerdote.

El sacerdote también hereda de `Character`, pero adicionalmente implementa la interfaz `Healable`.

Su función principal es apoyar a otros personajes mediante habilidades de curación.

Método:

`heal(Character target)`

La cantidad de salud recuperada dependerá de las características del sacerdote y de las reglas definidas por el estudiante.

5. Interfaces

El proyecto deberá utilizar interfaces para representar **habilidades adicionales** que no necesariamente pertenecen a todos los personajes.

Además de Healable, se deberán implementar al menos tres interfaces adicionales.

Healable

Representa la capacidad de curar a otro personaje.

Método:

heal(Character target)

Implementación principal:

Priest

MagicCaster

Representa la capacidad de utilizar habilidades mágicas.

Métodos posibles:

- castSpell(Character target)
- hasMana()

Implementación sugerida:

Mage

RangedAttacker

Representa la capacidad de realizar ataques a distancia.

Métodos posibles:

- rangedAttack(Character target)
- hasAmmunition()

Implementación sugerida:

Archer

Lootable

Representa la capacidad de obtener objetos o recompensas.

Métodos posibles:

- receiveItem(Item item)
- receiveGold(int amount)

Esta interfaz podría ser implementada por Character o por determinados tipos de personajes según el diseño del estudiante.

Propósito de las interfaces

El objetivo es demostrar que una característica no necesariamente tiene que estar ligada a la jerarquía de herencia.

Por ejemplo, un personaje podría ser simultáneamente:

- Character
- MagicCaster
- Healable

Esto permite combinar comportamientos mediante **composición de capacidades**, en lugar de crear una gran cantidad de subclases.

6. Inventario y composición

Cada Character deberá poseer un objeto Inventory.

La relación deberá representar **composición**:

Un personaje tiene un inventario y el inventario forma parte de la estructura del personaje.

La composición deberá reflejarse en el diseño de las clases. El inventario pertenece al personaje y su ciclo de vida está asociado a él.

Clase Inventory

La clase Inventory deberá contener información como:

Atributos

- capacity — capacidad máxima de objetos.
- items — colección de objetos.

- gold — cantidad de oro.
- weight — peso actual del inventario.
- maxWeight — peso máximo permitido.

Métodos principales

- addItem(Item item)
- removeItem(Item item)
- findItem(String name)
- showItems()
- calculateWeight()
- isFull()
- addGold(int amount)
- removeGold(int amount)

El estudiante deberá establecer reglas para determinar cuándo un objeto puede ser agregado al inventario.

Por ejemplo:

- No superar la capacidad máxima.
- No superar el peso máximo.
- No permitir objetos inexistentes.
- No permitir cantidades negativas.

7. Clase abstracta Item

Para ampliar el modelo del inventario, se deberá crear una clase abstracta:

Item

Esta clase representará cualquier objeto que pueda almacenarse en un inventario.

Atributos sugeridos

- id
- name
- description
- weight
- value

Métodos

- use(Character character)
- showInfo()

El método use() podrá ser abstracto para que cada tipo de objeto defina su comportamiento.

8. Tipos de objetos

A partir de Item se deberán crear diferentes tipos de objetos.

Además de Weapon y Armor, se deberán agregar al menos tres tipos adicionales.

Weapon

Representa un arma.

Atributos adicionales:

- damage
- durability
- weaponType

Métodos:

- use(Character character)
- calculateDamage()

Armor

Representa una pieza de armadura.

Atributos adicionales:

- defense
- durability
- armorType

Métodos:

- use(Character character)
- calculateDefense()

Potion

Representa una poción consumible.

Atributos:

- effectType
- effectValue

Ejemplos:

- Poción de salud.
- Poción de maná.
- Poción de fuerza.

Método:

use(Character character)

Accessory

Representa accesorios que pueden modificar las características de un personaje.

Ejemplos:

- Anillo.
- Amuleto.
- Collar.

Atributos posibles:

- bonusHealth
- bonusMana
- bonusStrength
- bonusDefense

Consumable

Representa objetos de un solo uso que producen algún efecto sobre el personaje.

Ejemplos:

- Comida.
- Antídoto.
- Pergamino.
- Elixir.

Método:

use(Character character)

9. Polimorfismo

El proyecto deberá demostrar explícitamente el uso de **polimorfismo**.

Por ejemplo, se podrá utilizar una colección:

List<Character>

que contenga objetos de diferentes tipos:

- Warrior
- Mage
- Archer
- Priest

Posteriormente, el programa podrá recorrer la colección y ejecutar:

```
character.attack(target);
```

sin necesidad de conocer el tipo concreto del personaje.

Cada objeto ejecutará su propia implementación de attack().

También deberá demostrarse polimorfismo mediante los objetos del inventario:

List<Item>

que puede contener:

- Weapon
- Armor
- Potion
- Accessory
- Consumable

Al ejecutar:

```
item.use(character);
```

cada tipo de objeto deberá comportarse de acuerdo con su implementación.

10. Sistema de misiones: Quest

El juego deberá contar con una clase denominada:

Quest

Una misión representa una actividad que puede ser asignada a uno o varios personajes.

La misión **no depende del ciclo de vida de los personajes**, por lo que representa una relación de **agregación**.

Una misión puede existir antes de que se asignen personajes y puede continuar existiendo aunque alguno de los personajes deje de participar.

Atributos de Quest

Como mínimo deberá contener:

- id
- name
- description
- difficulty
- status
- experienceReward
- goldReward
- participants
- objectives

Difficulty

Se recomienda utilizar un enum:

EASY, NORMAL, HARD, EPIC

Status

Se recomienda utilizar otro enum:

AVAILABLE, IN_PROGRESS, COMPLETED, FAILED

11. Objetivos de una misión

Para hacer más completo el modelo, Quest podrá contener una colección de objetivos.

Se podrá crear una clase:

QuestObjective

Atributos

- description
- targetAmount
- currentAmount
- completed

Métodos

- updateProgress(int amount)
- isCompleted()
- showProgress()

Una misión podría contener varios objetivos.

Ejemplo conceptual:

Quest: El Castillo Perdido

Objetivos:

1. Derrotar 5 enemigos.
2. Encontrar la llave del castillo.
3. Recuperar el tesoro.
4. Regresar al pueblo.

La misión podrá considerarse completada cuando todos sus objetivos estén terminados.

12. Agregación entre Quest y Character

La clase Quest deberá mantener una colección de personajes participantes.

Por ejemplo:

List<Character> participants

Esta relación representa **agregación**, ya que:

- Una misión puede tener varios personajes.
- Un personaje puede participar en varias misiones.
- Los personajes existen independientemente de las misiones.
- La misión puede existir sin participantes.
- Eliminar una misión no debería eliminar a los personajes.
- Eliminar un personaje de una misión no debería eliminar la misión.

La clase Quest deberá proporcionar métodos como:

- addParticipant(Character character)
- removeParticipant(Character character)
- showParticipants()
- addObjective(QuestObjective objective)
- updateQuestStatus()
- isCompleted()
- completeQuest()
- failQuest()
- giveRewards()

Clase de prueba / Main

El proyecto deberá incluir una clase Main que permita demostrar el funcionamiento del sistema.

La ejecución deberá mostrar, como mínimo:

1. Creación de personajes.
2. Creación de objetos.
3. Asignación de objetos a inventarios.
4. Equipamiento de armas o armaduras.
5. Ataques entre personajes.
6. Uso de habilidades especiales.
7. Curación de un personaje.
8. Creación de una misión.
9. Asignación de varios personajes a la misión.
10. Creación y actualización de objetivos.
11. Obtención de recompensas.
12. Demostración de polimorfismo.
13. Demostración de composición.
14. Demostración de agregación.